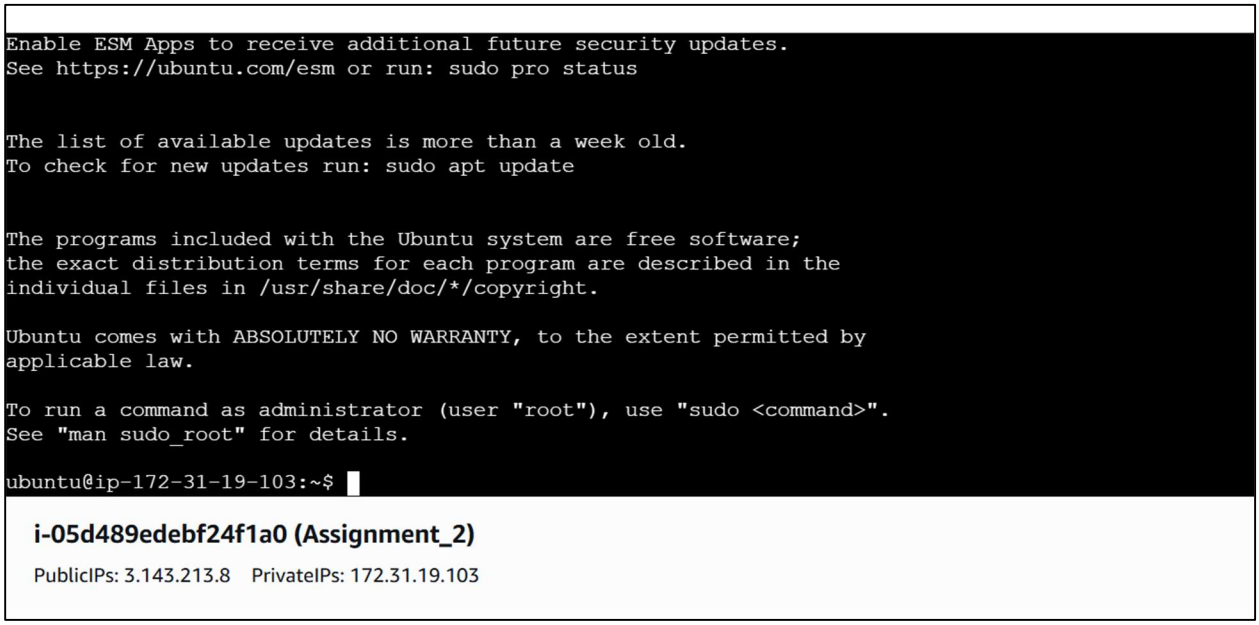
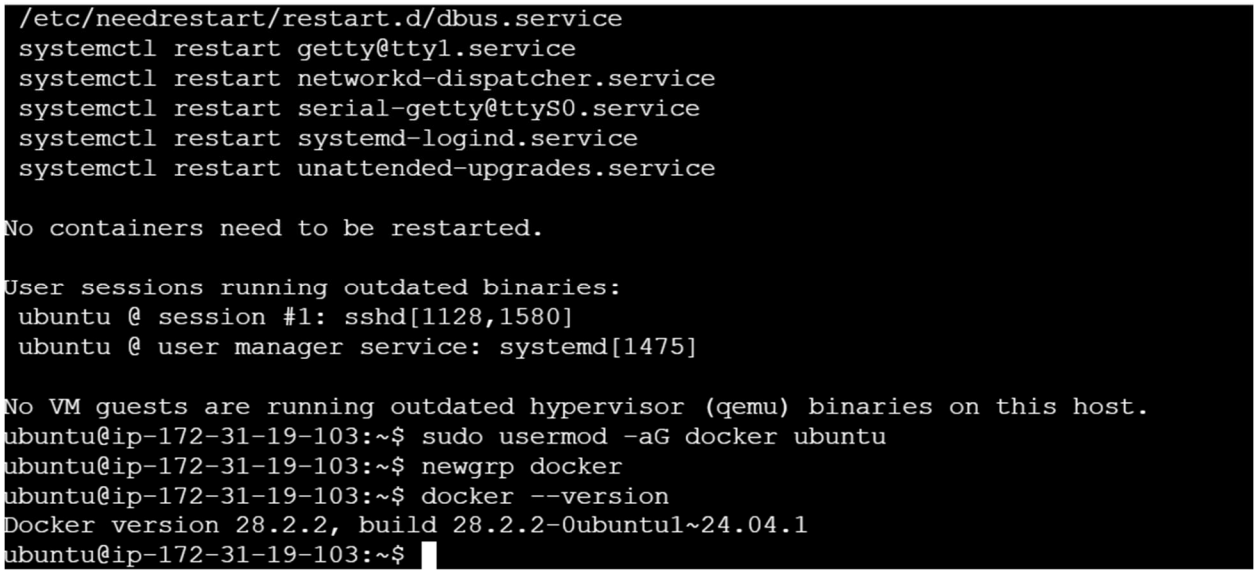
Table of Contents

Step 1: Updated System and Installed Docker

- Inside the SSH terminal, I first updated all system packages and then installed Docker using the following commands.
- After installation, I verified it by running.
- This confirmed that Docker was successfully installed and operational on my EC2 instance.





Step 2: Created Project Folder and Created Application and Docker Files

- To keep my files organized, I created a new directory for the project and moved into it.
- All subsequent application and configuration files were created inside this myapp directory.
- Next, I created the **Node.js web application** directly inside the EC2 shell using the `nano` editor.
- Created package.json
- This defined the Node.js project and specified dependencies for Express (the web framework) and MySQL2 (the database connector).
- Created index.js.
- To containerize the Node.js app, I created a Dockerfile using:
- This Dockerfile uses a lightweight Node.js image, installs dependencies, copies the source code, exposes port 3000, and starts the application using npm start.
- Since I was using Docker Swarm to manage secrets, I created a docker-stack.yml file to define both the **web** and **database** services.
- I made sure to escape $ as $$ inside the command line so Docker Compose would not try to interpolate them prematurely.
- This stack file defines both containers, attaches secrets, and specifies a persistent volume for MySQL data.

```
ubuntu@ip-172-31-19-103:~$ mkdir myapp
ubuntu@ip-172-31-19-103:~$ cd myapp
ubuntu@ip-172-31-19-103:~/myapp$ nano package.json
ubuntu@ip-172-31-19-103:~/myapp$ nano index.js
ubuntu@ip-172-31-19-103:~/myapp$ nano Dockerfile
ubuntu@ip-172-31-19-103:~/myapp$ nano docker-stack.yml
ubuntu@ip-172-31-19-103:~/myapp$
```

Package.json

```
{
  "name": "simple-web-app",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {
    "express": "^4.18.2",
    "mysql2": "^3.3.0"
  },
  "scripts": {
    "start": "node index.js"
  }
}

^G Help          ^O Write Out    ^W Where Is     ^K Cut          ^T Execut
^X Exit          ^R Read File    ^\ Replace      ^U Paste        ^J Justif
```

Index.js

```
  GNU nano 7.2                                                  index.js *
const express = require('express');
const mysql = require('mysql2/promise');
const fs = require('fs');

const app = express();
const PORT = process.env.PORT || 3000;

async function getDbConfig() {
  const readSecret = (name) => {
    try {
      return fs.readFileSync(`/run/secrets/${name}`, 'utf8').trim();
    } catch (e) {
      return process.env[name.toUpperCase()] || '';
    }
  };

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/
```

Dockerfile:

```
  GNU nano 7.2                                                        Docke
FROM node:18-alpine
WORKDIR /usr/src/app
COPY package.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Docker-stack.yml:

```
  GNU nano 7.2                                                  docker-stack.
version: "3.7"

services:
  web:
    image: zainabq055/simple-web-app:latest
    ports:
      - target: 3000
        published: 80
        protocol: tcp
        mode: host
    depends_on:
      - db
    environment:
      - DB_HOST=db
    secrets:
                                                                [ Wrote 50 lines
```

Step 3: Initialized Docker Swarm

- To enable Docker Secrets and use stack deployment, I initialized Docker Swarm.
- The output confirmed that my EC2 instance was now acting as a Swarm **manager node**.

```
ubuntu@ip-172-31-19-103:~/myapp$ docker swarm init --advertise-addr $(hostname -I | awk '{print $1}')
Swarm initialized: current node (t8osedmm0n07k2da4xuzlo8qr) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3ndpkx4nayicpjxots6cc4x2e1ne6fqf2vgu3vp1gzwjw409je-1oaz55583uqdogtlwgd6sxv

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
ubuntu@ip-172-31-19-103:~/myapp$ docker node ls
ID                            HOSTNAME          STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
t8osedmm0n07k2da4xuzlo8qr *   ip-172-31-19-103   Ready     Active         Leader           28.2.2
```

Step 4: Created Docker Secrets

- I then securely created all required secrets for database credentials.
- To verify that the secrets were created successfully, I ran: **docker secret ls**
- This displayed all four secrets.
- Docker secrets ensure that sensitive credentials are never stored directly in images or environment variables.

```
ubuntu@ip-172-31-19-103:~/myapp$ echo "appdb" | docker secret create db_name -
dmq6fqarnc96c8t671dledfxm
ubuntu@ip-172-31-19-103:~/myapp$ echo "appuser" | docker secret create db_user -
4x6zozvm1g2oprg97soj20rga
ubuntu@ip-172-31-19-103:~/myapp$ echo "verystrongpass" | docker secret create db_password -
0svk1ypqc0t3a90k7g4bkdk2g
ubuntu@ip-172-31-19-103:~/myapp$ echo "rootstrongpass" | docker secret create mysql_root_password -
jpj46pva7ml7darmq7k25j3ts
ubuntu@ip-172-31-19-103:~/myapp$ docker secret ls
ID                          NAME                DRIVER    CREATED          UPDATED
dmq6fqarnc96c8t671dledfxm   db_name                       58 seconds ago   58 seconds ago
0svk1ypqc0t3a90k7g4bkdk2g   db_password                   34 seconds ago   34 seconds ago
4x6zozvm1g2oprg97soj20rga   db_user                       46 seconds ago   46 seconds ago
jpj46pva7ml7darmq7k25j3ts   mysql_root_password           22 seconds ago   22 seconds ago
```

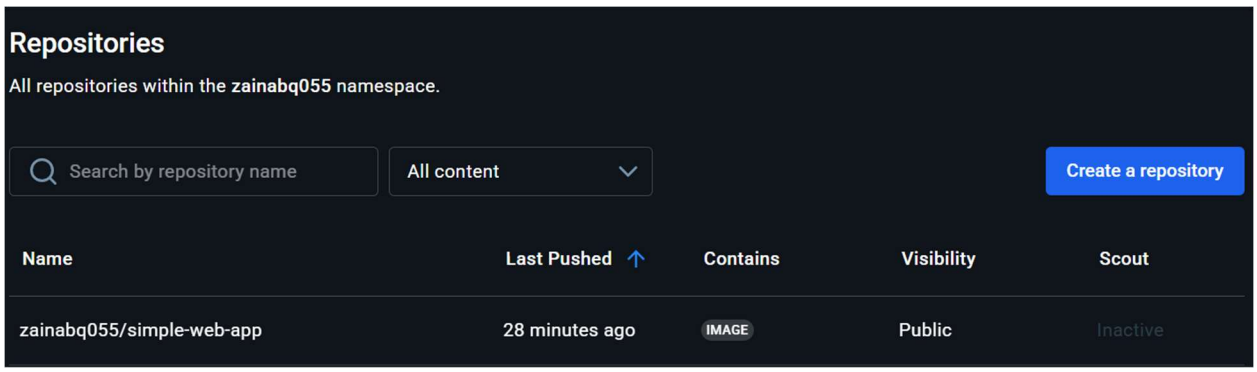
Step 5: Built and Pushed the Docker Image

- Next, I built the Docker image for my application.
- After the build completed, I logged into Docker Hub and pushed the image:
- This made the image available in my Docker Hub repository so it could be pulled automatically during deployment.

```
ubuntu@ip-172-31-19-103:~/myapp$ docker build -t zainabq055/simple-web-app:latest .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  7.168kB
Step 1/7 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
f18232174bc9: Verifying Checksum
f18232174bc9: Download complete
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
25ff2da83641: Verifying Checksum
25ff2da83641: Download complete
```

```
ubuntu@ip-172-31-19-103:~/myapp$ docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'


Your one-time device confirmation code is: NLQW-NFSX
Press ENTER to open your browser or submit your device code here: https://login.docker.com/ac

Waiting for authentication in the browser…


WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

```
ubuntu@ip-172-31-19-103:~/myapp$ docker push zainabq055/simple-web-app:latest
The push refers to repository [docker.io/zainabq055/simple-web-app]
13c63868c3c5: Pushed
c2196c9f24c9: Pushed
dcc21108aa64: Pushed
b11aadf332b5: Pushed
82140d9a70a7: Mounted from library/node
f3b40b0cdb1c: Mounted from library/node
0b1f26057bd0: Mounted from library/node
08000c18d16d: Mounted from library/node
latest: digest: sha256:c8d659f94af2f9bb28100cdccdc1111428751d845e5f70c27d4147608f7d069e size: 1991
```



Step 6: Deployed the Stack

- Once everything was ready, I deployed the stack.
- This created both the web and db services under a single stack called **myapp**.
- To confirm, I ran:

docker service ls
docker stack ps myapp

- Both services showed as "1/1" running, indicating that the containers were successfully deployed.

```
ubuntu@ip-172-31-19-103:~/myapp$ docker stack deploy -c docker-stack.yml myapp
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network myapp_default
Creating service myapp_web
Creating service myapp_db
```

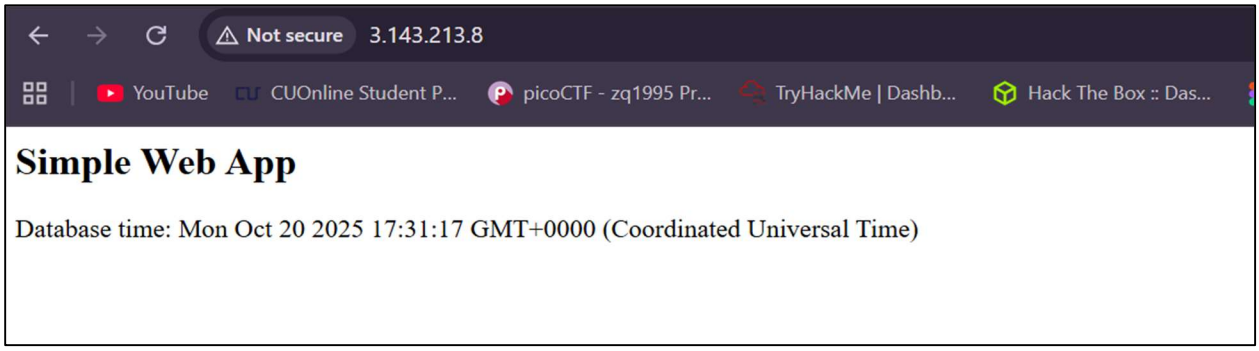
```
ubuntu@ip-172-31-19-103:~/myapp$ docker stack ls
NAME      SERVICES
myapp     2
ubuntu@ip-172-31-19-103:~/myapp$ docker stack ps myapp
ID             NAME          IMAGE                              NODE               DESIRED STATE   CURREN
za9djxf95kmg   myapp_db.1    mysql:5.7                          ip-172-31-19-103   Running         Runnin
ozy0ixiv877o   myapp_web.1   zainabq055/simple-web-app:latest   ip-172-31-19-103   Running         Runnin
3000/tcp,*:80->3000/tcp
```

Step 7: Tested the Application

- I copied my EC2 instance's **Public IPv4 address** from the AWS Console and opened it in a browser:

Error! Hyperlink reference not valid.

- The application loaded successfully and displayed:
- This confirmed that the Node.js application was running and connected to the MySQL database using the secrets I had configured.



Step 8: Verified Persistent Volume

- To ensure database data persistence, I checked the volumes using: **docker volume ls.**
- The output showed a named volume: **myapp_db_data.**
- This verified that MySQL data was stored in a persistent Docker volume, so even if the container is redeployed or restarted, the data remains intact.

```
ubuntu@ip-172-31-19-103:~/myapp$ docker volume ls
DRIVER    VOLUME NAME
local     myapp_db_data
```

Appendix

Docker-stack.yml:

```
version: "3.7"

services:
  web:
    image: zainabq055/simple-web-app:latest
    ports:
      - target: 3000
        published: 80
        protocol: tcp
        mode: host
    depends_on:
      - db
    environment:
      - DB_HOST=db
    secrets:
      - db_user
      - db_password
      - db_name
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure

  db:
    image: mysql:5.7
    command: ["bash", "-c", "export MYSQL_ROOT_PASSWORD=$$(cat
/run/secrets/mysql_root_password) && export MYSQL_DATABASE=$$(cat /run/secrets/db_name) &&
export MYSQL_USER=$$(cat /run/secrets/db_user) && export MYSQL_PASSWORD=$$(cat
/run/secrets/db_password) && exec docker-entrypoint.sh mysqld"]
    secrets:
      - mysql_root_password
      - db_name
      - db_user
      - db_password
    volumes:
      - db_data:/var/lib/mysql
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure
```

```yaml
secrets:
  db_name:
    external: true
  db_user:
    external: true
  db_password:
    external: true
  mysql_root_password:
    external: true

volumes:
  db_data:
```